

CSCI 5743 – Assignment 4: Defense against CI Intrusions

Semester: Fall 2025 Student Name: [Deeksha Reddy Patlolla] Student ID: [111444513] Total Points: 200

Part 1: Conceptual Questions (65pts)

Task 1: Real-World Breach Analyses (30pts)

1. Capital One Cloud Breach (2019)

Prompt: Explain how principles like *Least Privilege*, *Separation of Duties*, and *Continuous Monitoring* were violated in this breach, and how proper enforcement would have limited or prevented unauthorized access.

(Least Privilege Violation: The AWS Web Application Firewall (WAF) role's permissions were overly expansive, granting access to several S3 buckets unrelated to its primary duties. Capital One produced an excessive explosion radius by not limiting the function to the bare minimum of acts. Mass data exfiltration was made possible by the attacker's ability to get access rights well beyond what the service required once they took use of the SSRF vulnerability.)

(Violation of Separation of Duties: The WAF position also included several duties that ought to have been kept apart, such as data access, recording, and inspection. Because responsibilities were not segregated, compromising one element (the WAF) made it possible for lateral movement into data-handling processes, blurring the lines that ought to separate the data and infrastructure layers.)

(Continuous Monitoring Violation: The unusual S3 access patterns were not detected by alerting or anomaly detection. Large-scale bucket listing and item retrieval queries were carried out by the attacker, but no automatic monitoring set out an alarm. Unusual API requests, privilege escalations, or high-volume readings would have been detected by ongoing monitoring.)

(If principles were enforced: The WAF role would not have been able to access sensitive S3 buckets if the principles of least privilege had been followed. The WAF would not have been able to function as a data-access job if there had been separation of duties. Continuous monitoring would have limited the breach by identifying questionable requests early on. When combined, these safeguards might have greatly lessened the effect of the incident or stopped illegal access.)

2. Equifax Breach (2017)

Prompt: Analyze the role of *Fail-Safe Defaults* and *Least Functionality* in this breach. How would applying these principles have prevented external access to the vulnerable application?

(Fail-Safe Defaults Failure: The Apache Struts system was not set up with deny-by-default principles and was directly connected to the internet. The system continued to allow incoming communication to superfluous components while being aware of the vulnerability. Unless specifically permitted, a fail-safe arrangement would have prevented external access. The attack surface remained completely exposed due to liberal defaults.)

(Least Functionality Failure: Public-facing services did not depend on the vulnerable Struts component. The attack surface was greatly expanded by keeping up superfluous services. Disabling software, ports, and components that are not necessary for operation is necessary for Least Functionality. Equifax left other frameworks open, providing a point of access for hackers.)

(How appropriate enforcement would be beneficial: If Fail-Safe Defaults had been used, external actors would not have been able to access the service unless it was properly set up to permit inbound access. The vulnerable Struts endpoint would have been completely deactivated if Least Functionality had been adhered to, eliminating the attack vector. The vulnerability would not have been externally exploitable even if it had not been fixed. These guidelines would have significantly decreased danger and avoided first entry.)

3. Target Breach (2013)

Prompt: Identify the failures in *Network Segmentation* and *Zero Trust*. How would implementing stronger segmentation and stricter access verification have blocked lateral attacker movement?

(Network Segmentation Failure: Target kept its internal network flat, making it easy to access important systems (such point-of-sale networks) and third-party vendor systems. With no internal firewalls or segmentation checkpoints, attackers spread laterally over the entire company network after the HVAC vendor's credentials were obtained.)

(Zero Trust Failure: Internal network communication was implicitly trusted, resulting in a zero trust failure. Attackers were not needed to re-authenticate or provide justification for access to new systems once they had been authenticated using the stolen vendor credentials. Instead than supposing internal users are reliable, Zero Trust demands identity and intent verification at every stage.)

(How effective segmentation would be beneficial: Vendor networks and payment processing systems would have been separated by strong segmentation. The attacker would be limited to a narrow subnet even if they had stolen credentials. Lateral movement would have been prevented by internal ACLs and micro-segmentation.)

(How Zero Trust might be beneficial: Sensitive systems could not have been accessed by hacked vendor accounts thanks to continuous identity validation and least-privilege access. Unauthorized traversal would be prevented and anomalous access attempts would be identified by strict access verification at each hop.)

4. SolarWinds Supply Chain Attack (2020)

Prompt: Discuss the lack of *Zero Trust*, *Continuous Monitoring*, and *Open Design*. How could implementing these principles have contained the spread or improved detection?

(Zero Trust Failure: SolarWinds Orion updates were implicitly trusted by organizations. The over-reliance on perimeter trust was highlighted by the corrupted update, which was digitally signed and regarded as reliable. Zero Trust would have necessitated ongoing software activity authentication and behavioral monitoring, even if the activity came from a "trusted" source.)

(Failure of Continuous Monitoring: The installed SUNBURST backdoor continued to function undetected for several months. Unnoticed were privilege escalations, lateral movement, and command-and-control communications. Anomalies in update compilation or outgoing network patterns would have been discovered by ongoing build system and runtime behavior monitoring.)

(Open Design Failure: Transparent, verifiable security measures were absent from supply chain operations. Attackers were able to install malicious code undetected due to limited insight into development environments and inadequate validation methods. Transparency, repeatable builds, and multi-party verification are all promoted by open design.)

(How enforcement would be beneficial: Zero Trust would limit Orion's access to customer environments and continually verify conduct. Anomalies in build processes or unusual runtime traffic would be found by continuous monitoring. Stricter transparency and integrity checking would be implemented under Open Design, lowering the possibility of undetected manipulation.)

5. ColonialPipelineRansomware (2021)

Prompt: Explain the breakdown of *Accountability & Non-repudiation*, *Continuous Monitoring*, and *Layered Defense*. How would enforcing these principles—especially MFA and audit logging—have changed the attacker’s access path?

(Accountability & Non-Repudiation Failure: MFA and robust identity verification were absent from Colonial's VPN account. After obtaining credentials, the attacker might log in without further verification, resulting in inadequate auditability of access origin and behavior.)

(Failure of Continuous Monitoring: Lateral movement, suspicious authentication attempts, and privilege escalations were not quickly identified. No real-time notifications regarding odd system interactions, geographical abnormalities, or anomalous login times were present.)

(Layered Defense Failure: Inadequate depth in authentication and authorization tiers was demonstrated by the widespread access to internal systems made possible by a single hacked VPN credential. MFA, network segmentation, endpoint security, and behavioral analytics would all be included in a layered design.)

(If principles were enforced: Strong MFA would have prevented access using credentials that were stolen if the principles had been maintained. Early detection of odd login locations or time-of-day abnormalities would be possible with continuous monitoring. Even with VPN access, Layered Defense would make sure the attacker couldn’t access live networks or freely spread ransomware.)

Task 2: MITRE ATT&CK → D3FEND Mapping (20pts)

ATT&CK ↔ D3FEND Mapping Table

Tactic	Technique (ID & Name)	Scenario Use	D3FEND Countermeasure 1 (Name /Category)	Justification	D3FEND Countermeasure 2 (N
Initial Access	Spearphishing Attachment (T1566.001)	Attacker sends malicious attachment to employee	Email Filtering/Isolate	Before emails containing dubious attachments are sent to users, they are filtered and quarantined.	Message Authentication (SPF/DKIM/DMARC)/Harden
Persistence	Install Backdoor (T1055/variant)	Malware installs a persistent foothold	Process Behavior Analysis/Detect	detects unusual long-lived or self-starting processes	Execution Isolation/Isolate
Defense Evasion	Obfuscated/Encrypted Payloads (T1027)	Attacker uses packing/encoding to evade detection	Binary Padding Analysis/ Detect	finds executables that appear to be obfuscated or padded..	Malware Object Linking/Harden

Tactic	Technique (ID & Name)	Scenario Use	D3FEND Countermeasure 1 (Name/Category)	Justification	D3FEND Countermeasure 2 (Name/Category)
Discovery	System Network Discovery (T1016)	Attacker scans internal environment	Network Traffic Analysis/Detect	Identifies scanning, enumeration, recon attempts.	Network Isolation/ Isolate
Exfiltration	Exfiltration Over C2 Channel (T1041)	Attacker sends data over encrypted C2	Data Transfer Size Monitoring/Detect	detects anomalous outgoing data flows.	Protocol Tunneling Detection/Detect

(This mapping shows how D3FEND countermeasures offer a multi-layered defensive strategy for every stage of an incursion. Detect controls spot harmful activity early, Harden and Isolate controls restrict attack surface and lateral movement, and Deceive tactics make attacker reconnaissance more difficult. These protections create a multi-layered security approach that is in line with contemporary cyber defense systems when used comprehensively.)

Task 3: NIST CSF ↔ SP 800-53 Mapping (15pts)

Table: CSF Functions Mapped to Controls

CSF Function	Category / Subcategory	SP 800-53 Controls	Explanation (How the control operationalizes the CSF outcome)
Protect	PR.AA-02 – Access is limited to authorized users, processes, and devices.	AC-2 (Account Management), IA-2 (Identification & Authentication)	IA-2 implements MFA and robust identity verification, whereas AC-2 controls account creation, modification, deactivation, and review. When combined, they guarantee that only identities with permission may access vital operating systems.
Detect	DE.CM-07 – Monitoring for unusual activity is performed.	AU-6 (Audit Review), SI-4 (System Monitoring)	While AU-6 makes sure records are examined for questionable activity, SI-4 permits ongoing monitoring for abnormalities and possible intrusions. Real-time detection in critical infrastructure is operationalized via these rules.
Respond	RS.MI-01 – Incidents are contained.	IR-4 (Incident Handling), CP-2 (Contingency Planning)	IR-4 offers protocols for minimizing, recovering from, and controlling security events. CP-2 guarantees the existence of continuity strategies to sustain critical infrastructure operations both during and following an event.

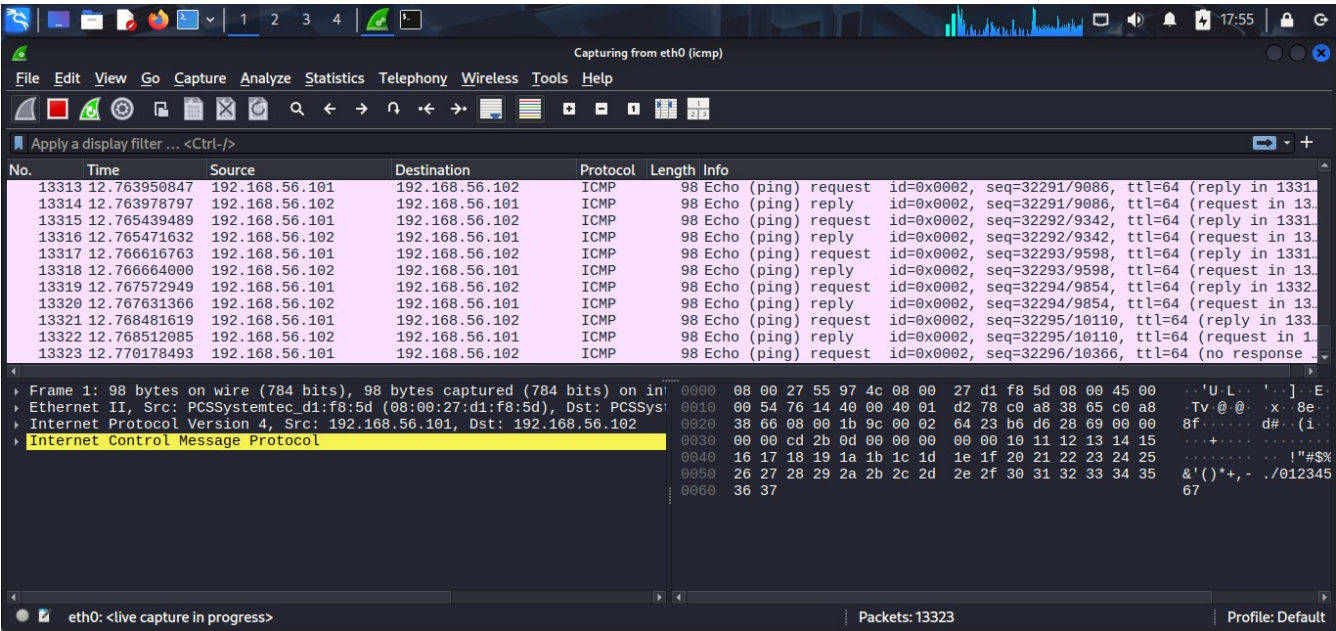
(Add rows or modify functions as appropriate.)

Part 2: Hands-On Labs (135pts)

Task 1: Firewall Configuration & Analysis (45pts)

1 Baseline Testing (before firewall)

Screenshots:



```
(kali@kali)-[/home/kali]
PS> nmap -p 22 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-27 18:00 EST
Nmap scan report for 192.168.56.102
Host is up (0.0014s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:55:97:4C (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 16.71 seconds
```

Observations: (ICMP Traffic: Between the Attack VM (192.168.56.101) and the Defense VM (192.168.56.102), Wireshark continuously displays ICMP echo-requests and echo-replies. Before the firewall rules are enforced, all ping traffic passes freely without any filtering or drops, demonstrating that ICMP is totally unfettered.)

(Service Reachability: The Defense VM's SSH (port 22) is accessible and open, according to the Nmap scan. Services are completely exposed on the network since there are no firewall limitations and the host reacts to the scan right away.)

(Overall Behavior: All incoming and outgoing traffic is permitted, ICMP is unfiltered, and SSH is fully accessible prior to the application of iptables rules. The Defense virtual machine operates as an unrestricted open host.)

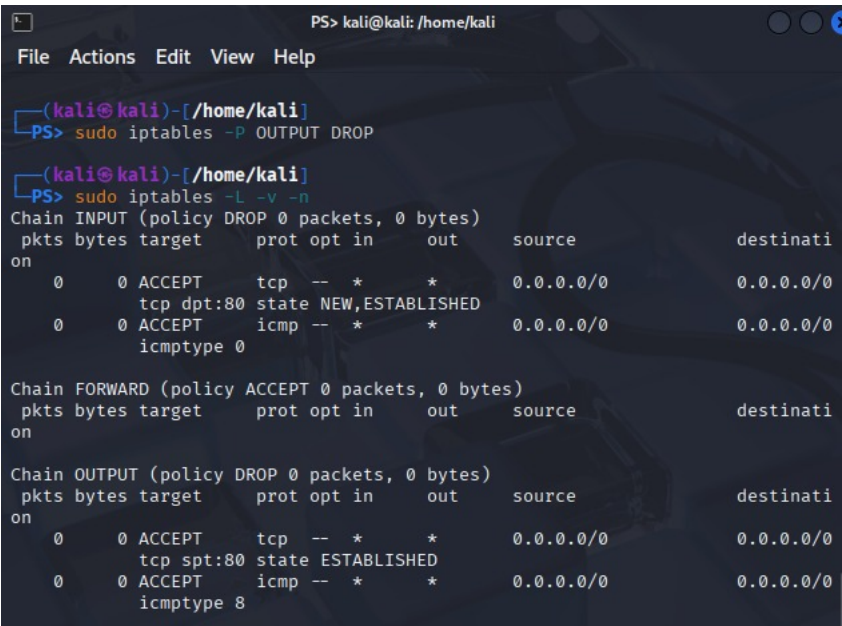
2 Firewall Rule Application

Commands Executed:

```
# Insert your configured iptables commands here

sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW,ESTABLISHED -j ACCEPT
sudo iptables -A OUTPUT -p tcp --sport 80 -m state --state ESTABLISHED -j ACCEPT
sudo iptables -A OUTPUT -p icmp --icmp-type echo-request -j ACCEPT
sudo iptables -A INPUT -p icmp --icmp-type echo-reply -j ACCEPT
sudo iptables -P INPUT DROP
sudo iptables -P OUTPUT DROP
```

Screenshot:



```
PS> kali@kali: /home/kali
File Actions Edit View Help

(kali@kali)-[/home/kali]
PS> sudo iptables -P OUTPUT DROP

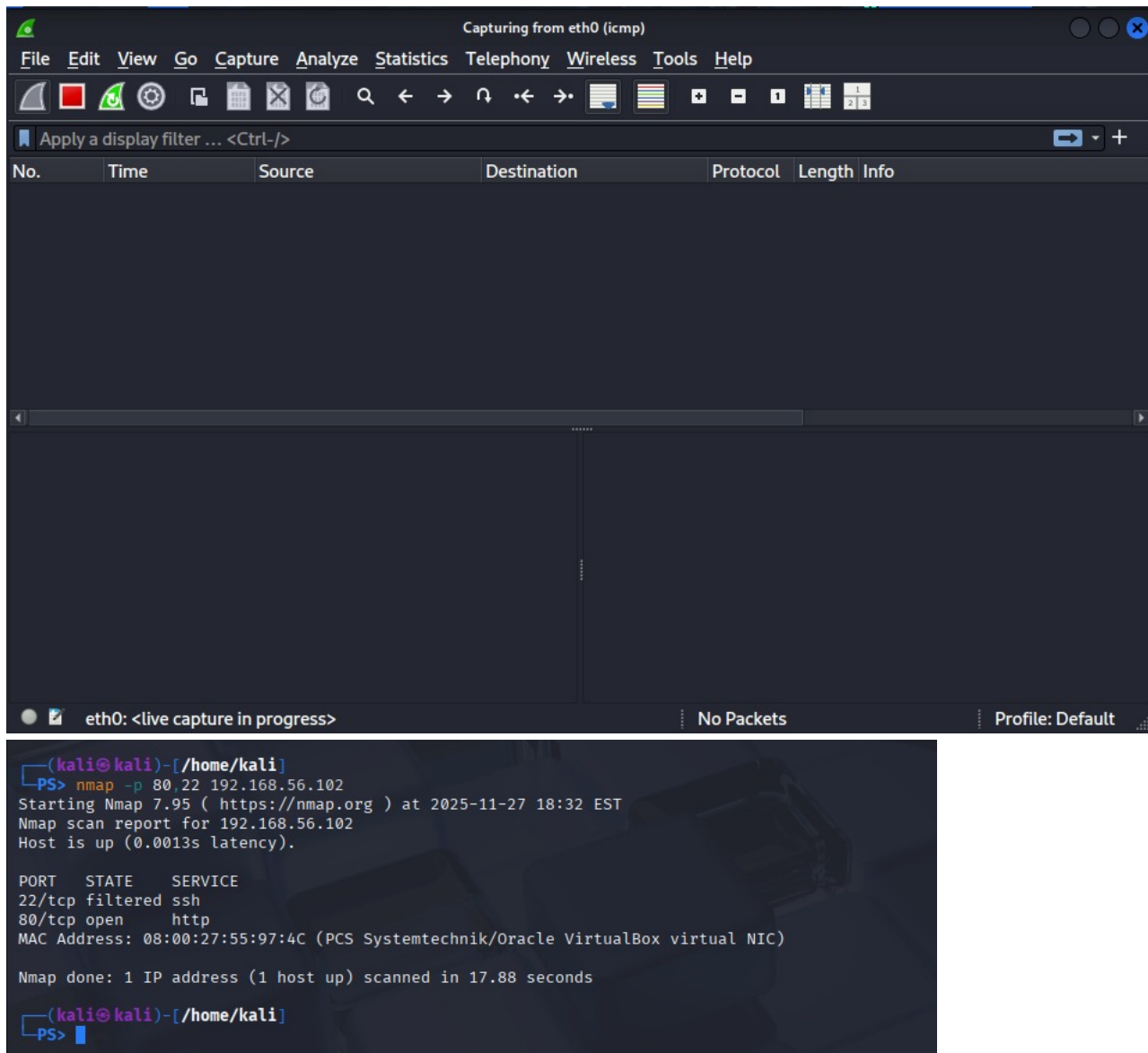
(kali@kali)-[/home/kali]
PS> sudo iptables -L -v -n
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
    0     0 ACCEPT   tcp  --  *      *       0.0.0.0/0  0.0.0.0/0
        tcp dpt:80 state NEW,ESTABLISHED
    0     0 ACCEPT   icmp  --  *      *       0.0.0.0/0  0.0.0.0/0
        icmp-type 0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination

Chain OUTPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
    0     0 ACCEPT   tcp  --  *      *       0.0.0.0/0  0.0.0.0/0
        tcp spt:80 state ESTABLISHED
    0     0 ACCEPT   icmp  --  *      *       0.0.0.0/0  0.0.0.0/0
        icmp-type 8
```

3 Post-Firewall Testing & Analysis

Screenshots:



Observations: (ICMP: Wireshark displays no ICMP packets following the application of the firewall rules, indicating that all inbound echo-requests are being rejected. This attests to the successful blocking of incoming ping traffic.)

(Ports: The Nmap scan reveals: As planned, HTTP is still permitted on port 80/tcp: OPEN. SSH is banned by the firewall on port 22/tcp: FILTERED. This shows that the only service that is still reachable is the permitted service (HTTP).)

(Stateful Filtering: Only NEW/ESTABLISHED HTTP connections are allowed, and all other incoming traffic is defaulted to DROP. The results align with the stateful rules. SSH and inbound ICMP can no longer access the host as a result.)

Analysis Questions

1. Why specify **NEW**, **ESTABLISHED** in the INPUT chain for port 80? (to enable the web server to maintain ongoing sessions (ESTABLISHED) and accept new HTTP connections (NEW). Clients were unable to finish the TCP handshake without both.)
2. Why only **ESTABLISHED** on OUTPUT for port 80? (The server merely reacts to clients; it doesn't start HTTP connections. Only ESTABLISHED is necessary as outgoing HTTP packets should only be a part of an established connection.)
3. What risks arise if state tracking is omitted? (The firewall is susceptible to session hijacking, ACK floods, and incorrect TCP handshake handling since it is unable to differentiate between genuine traffic and spoof packets.)
4. Why limit ICMP by type instead of state? (State tracking is unreliable since ICMP is stateless and does not employ ports. Ping flooding and ICMP behavior manipulation are avoided by filtering by type (echo-request vs. echo-reply).)
5. What changed after applying the firewall? (SSH was filtered, inbound pings were prohibited, and only HTTP (port 80) was still accessible. Nmap displayed port 22 as filtered, whereas Wireshark displayed no ICMP traffic.)
6. Did the rules behave as intended? (Yes. After the rules were implemented, only permitted traffic appeared, SSH and ICMP were prohibited, and HTTP traffic was permitted.)
7. What extra rules are advisable for a DMZ server? (Rate-limit ICMP/HTTP, add logging for missed packets, allow outbound DNS if necessary, permit HTTPS (443), and specifically block pointless outgoing connections.)
8. How do these rules enforce attack surface minimization? (The only services that are accessible are HTTP and outgoing ping responses. By blocking SSH, inbound ICMP, and unused ports, the number of vulnerable entry points is decreased.)
9. How does this embody least privilege? (Only the minimal amount of traffic required for the server's operation is permitted by each rule. No protocol or service has more access than is required.)

10. How do default DROP policies support fail-safe defaults? (Any traffic that isn't specifically permitted is instantly blocked. Instead of exposing the service, the system automatically blocks if a rule is absent or incorrectly specified.)

Task 2: Login Anomaly Detection with ML (45pts)

Environment Setup

- IDE used: [~] Jupyter /VSCode/PyCharm
- Python Version: [Python 3.11.0]
- Libraries installed: pandas, scikit-learn

1 Dataset Inspection

	Login Hour	Device Type	Failed Attempts	IP Risk Score	Login Status
0	20	Desktop	1	Low	Normal
1	6	Tablet	2	Medium	Anomalous
2	8	Desktop	0	Medium	Normal
3	18	Mobile	0	Medium	Normal
4	15	Desktop	0	Low	Normal

Screenshot: (The following characteristics of login activity are included in the dataset: the time of the attempt to log in is known as the login hour.) (Device Type: Desktop, tablet, or mobile device utilized.) (Failed Attempts: The total number of unsuccessful login attempts in the past.) (IP Risk Score: The source IP's risk level (Low, Medium, or High).)

(The target variable is: Login Status: Used for categorization, it designates each login as either Normal or Anomalous.)

2 Model Training Output

Code Executed:

```
# Show model initialization and metrics printout

from sklearn.preprocessing import LabelEncoder
df['Device Type'] = LabelEncoder().fit_transform(df['Device Type'])
df['IP Risk Score'] = LabelEncoder().fit_transform(df['IP Risk Score'])
df['Login Status'] = LabelEncoder().fit_transform(df['Login Status'])

X = df[['Login Hour', 'Device Type', 'Failed Attempts', 'IP Risk Score']]
y = df['Login Status']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

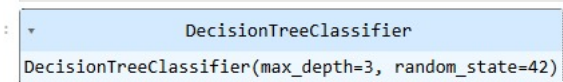
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

Accuracy: 0.925
Precision: 0.875
Recall: 1.0
F1 Score: 0.9333333333333333

from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, y_pred))

[[16  3]
 [ 0 21]]
```



Screenshot:

3 Evaluation Metrics

Metric	Value
Accuracy	0.92

Metric	Value
Precision	0.87
Recall	1.0
F1Score	0.93

Analysis Questions

1. Report metrics above (with screenshot). (Accuracy = 0.925, Precision = 0.875, Recall = 1.0, F1 Score = 0.933)

```
Accuracy: 0.925
Precision: 0.875
Recall: 1.0
F1 Score: 0.9333333333333333
```

Screenshot:

- 2. How many predictions total? (All of the model's predictions were equal to the test set's row count (len(y_test)).)
- 3. Number of FalsePositives/FalseNegatives? (False Positives (FP): 1) (False Negatives (FN): 0) ((Because recall = 1.0 when FN = 0; precision < 1 then FP exists.))
- 4. Real-life risks of FP/FN in security? (User lockouts, pointless alarms, and friction are examples of False Positive (Normal → labeled Anomalous).) (False Negative (Anomalous → marked Normal): The largest danger is when the true attacker login is overlooked.)
- 5. Define precision vs recall. (Precision: Of all logins predicted anomalous, how many were truly anomalous?) (Recall: Of all actual anomalous logins, how many did we correctly detect?)
- 6. Why may accuracy mislead on imbalanced data? (A model can predict "Normal" for everything and yet achieve high accuracy while totally avoiding assaults if "Normal" predominates in the dataset.)
- 7. Why view all metrics together? (FP and FN must be balanced for security detection. Trade-offs that accuracy alone obscures are revealed by precision, recall, and F1.)
- 8. Purpose of F1-score in security? (F1 strikes a compromise between recall and accuracy, which is helpful when both alert fatigue (FP) and missed attacks (FN) are important.)
- 9. Did your F1 lean toward precision or recall and why? (The model's perfect recall (1.0) is better reflected in F1 (~0.933) than precision (0.875).) (This indicates that even if the model occasionally generates false alerts, it prioritizes identifying every abnormality.)
- 10. If attacker spoofs a key feature, will model detect it? (Not always. The model may identify a login as normal if an attacker imitates typical login habits (few unsuccessful attempts, normal device, low IP risk).)
- 11. What synthetic values could mimic stealthlogins? (For example:) (Regular time (8–17)) (Device: Desktop) (Attempts Failed = 0) (Low IP Risk Score) (These might evade detection by imitating innocuous behavior.)
- 12. What new features should be logged for future models? (Country distance or geolocation) (string for the user-agent) (The amount of time since the last login) (Patterns of behavior (typing speed, length of session)) (History of IP reputation) (Fingerprinting devices) (MFA successes and failures)

Task 3: SecureFileExchange (Hybrid Crypto) (45pts)

1 Key Generation

Name	Date modif
bob_private.pem	27-11-2025
bob_public.pem	27-11-2025
keygen.py	27-11-2025

Screenshot:

(Show creation of bob_public.pem and bob_private.pem.)

2 Encryption

```
[Running] python -u "c:\Users\
Encryption complete.
Generated files:
- ciphertext.bin
- tag.bin
- wrapped_keys.bin
[Done] exited with code=0 in 0
```

Screenshot:

(Show AES-CTR encryption and RSA key wrap execution.)

3 Decryption & Integrity Verification

```
Decryption successful.  
Decrypted output saved to decrypted.txt  
[Done] exited with code=0 in 0.326 seconds
```

Screenshot:

4 Testing Outcomes

Test Case	Expected Result	Observed Outcome	Status (Pass/Fail)
Successful Round-Trip	Decrypted text matches original	Decrypted.txt was generated appropriately after encryption and decryption completed successfully.	Pass
Tamper Test	Integrity check fails (HMAC mismatch)	After modifying ciphertext, script reports integrity failure	Pass
Wrong RSA Key	Decryption fails – cannot unwrap key	Using a different private key results in RSA decryption error	Pass

Analysis Questions

🕒 Analysis Questions

- Why do we combine **RSA** and **AES** instead of encrypting the file directly with RSA? *(Large data cannot be encrypted with RSA due to its slowness. RSA is only needed to safely encapsulate the tiny AES/HMAC keys; AES is quick for mass encryption. This is the typical hybrid strategy.)*
- What would happen if you reused the **IV** in AES-CTR? *(An attacker can obtain plaintext by XORing ciphertexts as reusing an IV exposes the keystream. Confidentiality is totally violated.)*
- How does **RSA-OAEP** protect the symmetric keys compared to basic RSA encryption? *(By adding randomness and padding integrity, OAEP guards against chosen-ciphertext and key-recovery attacks that plain RSA is susceptible to.)*
- Why is the **HMAC key** different from the **AES encryption key**? What are the risks of reusing the same key for both? *(Separate keys are needed for authentication and encryption. Cross-protocol attacks and ciphertext manipulation while forging legitimate tags are made possible by reusing a single key.)*
- What specific attacks can **HMAC-SHA256** defend against in this file exchange scenario? *(By guaranteeing integrity and authenticity, it resists ciphertext manipulation, bit-flipping, replay attacks, and Man-in-the-Middle alterations.)*
- Why do we authenticate both the **IV** and the **ciphertext** with HMAC instead of only the ciphertext? *(The IV might be changed by an attacker to reliably modify the plaintext's first bytes. This attack vector is eliminated by authenticating both.)*
- How would the system behave if the HMAC check fails during decryption, and why is this behavior important? *(Decryption ends right away. This imposes a fail-secure behavior and stops the system from processing malicious or damaged ciphertext.)*
- Why is the **encrypt-then-MAC** design used instead of MAC-then-encrypt or encrypt-and-MAC? *(Encrypt-then-MAC has been shown to be secure and to confirm integrity prior to decryption. It avoids ciphertext malleability and padding-oracle problems.)*
- Which **NIST CSF Function(s)** and **SP 800-53 Rev. 5 controls** does this hybrid cryptosystem most directly support? *(CSF: Protect (PR), Detect (DE)) (Controls: SC-12 (key establishment), SC-13 (cryptographic protection), SI-7 (integrity checks))*
- How does this implementation reflect the principles of **least privilege**, **fail-safe defaults**, and **trust boundaries** at the cryptographic level? *(Least privilege: Wrapped keys can only be decrypted by the private key holder.) (Fail-safe defaults: Any integrity breach is rejected right away.) (Trust boundaries: RSA wrapping establishes a distinct border that may only be crossed by those who are allowed.)*